

《面向对象程序设计》之——

模 板

连 盛

shenglian@fzu.edu.cn

计算机与大数据学院

福州大学

回顾：面向过程，面向对象？



面向过程

- 定义函数

- 函数1
- 函数2
- 函数3
- 函数4

- 定义数据

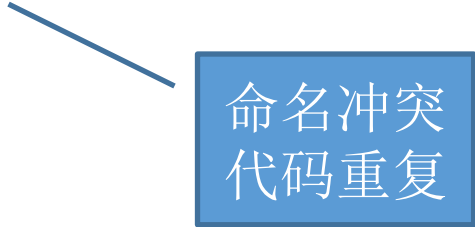
- 数据1
- 数据2
- 数据3
- 数据4

命名冲突
代码重复

- 各种函数、数据的操作

以计算器小程序为例

- 想象我们要做一个计算器的小程序。加减乘除函数要确保处理的是合理的数据，因此都要写对参数进行检测的代码。我们可以将这个“检测代码”独立出来，因此有如下函数：
- 检测，整数加，整数减，整数乘，整数除，复数加，复数减，复数乘，复数除，小数加.....



命名冲突
代码重复

以计算器小程序为例：类

```
整数 {  
    检测  
    加  
    减  
    乘  
    除  
}
```

```
复数 {  
    检测  
    加  
    减  
    乘  
    除  
}
```

```
小数 {  
    检测  
    加  
    减  
    乘  
    除  
}
```

.....



类

以计算器小程序为例：继承

```
数 {  
    检测  
    加  
    减  
    乘  
    除  
}
```

```
整数 {  
    沿用上面数的设计  
}
```

```
小数 {  
    沿用上面数的设计  
}
```

.....

有了类的继承，还能继续改进吗？

```
小数类 {  
    加  
    减  
    乘  
    除  
}
```

如果我想一个小数计算类有四舍五入功能，另一个不要，难道需要写两个类吗？



有了类的继承，还能继续改进吗？

小数类 {

标识变量：是否四舍五入

标识变量：是否限定小数点后位数

构造函数（设置上面的标识）

加（会根据上面两个标识变量输出不同结果）

减（会根据上面两个标识变量输出不同结果）

乘（会根据上面两个标识变量输出不同结果）

除（会根据上面两个标识变量输出不同结果）

}

以计算器小程序为例：模板

```
单整数类 {  
    单整数加  
    单整数减  
    单整数乘  
    单整数除  
}
```

```
双整数类 {  
    双整数加  
    双整数减  
    双整数乘  
    双整数除  
}
```



```
<变量类型>整数 {  
    <变量类型>加  
    <变量类型>减  
    <变量类型>乘  
    <变量类型>除  
}
```

栗子回顾：计算器小程序

- 面向过程：函数若干，数据若干（命名冲突，代码重复）
- 提取共性函数（检测）
- 归“类”（整数、复数、小数等）
- 继续提取/抽象，得到父类、子类，为继承关系
- 模板功能
- 实例，对象，构造函数，析构函数，重载，多态.....

讲授内容

- 类属机制
- 函数模板的定义与使用
- 类模板的定义与使用

例子1: 函数模板例子

```
template < class   ElementType >
void sortArray(ElementType b[],
               int len)
{for (int pass = 0;
    pass < len - 1; pass ++ )
  for(int i = pass+1; i <= len-1; i++)
    if ( b[ pass ] > b[ i ] )
    {ElementType hold;
     hold = b[ pass ];
     b[ pass ] = b[ i ];
     b[ i ] = hold;
    }
}
```

类属机制 (1/2)

- 从功能来看，有些程序非常相似，只是处理对象（数据）的类型不同

- 比如前述的程序

```
void sortArray(int b[],int len);
```

```
void sortArray(double b[],int len);
```

- 可以考虑提供具有相同逻辑功能的程序，而将数据类型作为参数——类属编程

- C++中的模板：类模板和函数模板

类属机制 (2/2)

- 更好的代码重用性
 - 重用源代码
- 用户使用模板时，编译器替换模板参数（类型）

函数模板 (1/5)

- 对不同类型的数据执行相似的操作
 - 函数重载
 - 函数模板
- 函数重载——代码冗余
 - 增加编程负担
 - 程序的维护问题（如功能修改, 算法变动）

函数模板 (2/5)

- 函数模板为所有的函数提供唯一的一段函数代码
 - 置换代码中的类型参数得到模板函数——实例化
 - 函数模板不是函数，不能被执行
 - 实例化后的模板函数是真正的函数，可以被执行



函数模板 (3/5)

- 函数模板的定义都是以关键字 `template` 开头
- `template` 之后 `< >` 中是函数模板的参数列表
- 函数模板的参数称为类型参数，其类型为 `class` 或 `typename`，如

```
template <class T>
```

```
template <class T1, typename T2>
```

函数模板 (4/5)

- `<...>`之后是模板的函数定义，可以将类型参数用于函数返回值、参数或局部变量的类型声明
- 每个类型参数名要在函数的参数列表中至少出现一次
- 类型参数名的作用域限于函数模板的范围内

例子1: 函数模板例子

```
template < class   ElementType >
void sortArray(ElementType b[], int len)
{for (int pass = 0;
    pass < len - 1; pass ++ )
  for(int i = pass+1; i <= len-1; i++)
    if ( b[ pass ] > b[ i ] )
    {ElementType hold;
     hold = b[ pass ];
     b[ pass ] = b[ i ];
     b[ i ] = hold;
    }
}
```

函数模板 (5/5)

- 函数模板规定了对数据的处理流程
- 某些数据类型（类型参数）要等到模板实例化时再确定具体的类型
- 函数模板的实例化由编译器来完成
 - 根据函数调用的实参类型确定类型参数的具体类型
 - 用具体类型替换模板的函数定义中所出现的所有类型参数，完成函数模板的实例化

例子2: 使用函数模板实现数组的排序和输出 (1/4)



```
#include <iostream>
#define SIZE 8
template <class ElementType >
void sortArray(ElementType b[], int len)
{for (int pass=0; pass < len-1; pass++)
    for(int i = pass+1; i <= len-1; i++)
        if (b[pass] > b[i])
        {ElementType hold;
         hold = b[ pass ];
         b[ pass ] = b[ i ];
         b[ i ] = hold;
        }
}
```

例子2: 使用函数模板实现数组的排序和输出 (2/4)



```
template <class ElementType >
void displayArray(ElementType b[],
                  int len)
{for(int index=0;
    index <= len-1; index++)
    if ( index != len -1 )
        cout << b[ index ] << "\t";
    else
        cout << b[ index ] << endl;
}
```

例子2: 使用函数模板实现数组的排序和输出 (3/4)



```
main()
```

```
{int ai[SIZE]={18,35,36,61,9,112,77,12};  
  double af[SIZE]={12.1,-23.8,3.7,-16.0,  
                  9.1,12.12,7.7,56.3};  
  cout << "Before sorting:\n";  
  cout << "ai: \t";  
  displayArray(ai, SIZE);  
  sortArray(ai, SIZE);  
  cout << "After sorting:\n";  
  cout << "ai: \t";  
  displayArray(ai, SIZE);  
  //接下页
```

例子2: 使用函数模板实现数组的排序和输出 (4/4)



```
cout << "Before sorting:\n";  
cout << "af: \t";  
displayArray (af, SIZE) ;  
sortArray (af, SIZE) ;  
cout << "After sorting:\n";  
cout << "af: \t";  
displayArray (af, SIZE) ;  
return 0 ;  
}
```


程序运行结果

Before sorting:

ai: 18 35 36 61 9 112 77 12

After sorting:

ai: 9 12 18 35 36 61 77 112

Before sorting:

af: 12.1 -23.8 3.7 -16 9.1 12.12 7.7 56.3

After sorting:

af: -23.8 -16 3.7 7.7 9.1 12.1 12.12 56.3

类模板

- 类模板：将类定义中的数据类型参数化
- 类模板的实例化：用具体的数据类型替换类型参数以得到具体的类（模板类）
- 模板类可用于声明对象（注意格式）



例子3: 用类模板实现栈(1/6)

```
// tstack.h: 类模板的定义
template < typename ElementType >
class Stack
{public:
    Stack( int = 8 ); // 默认栈元素数目为8
    ~Stack() { delete [] data; };
    int pop(ElementType &num);
    int push(ElementType num);
private:
    ElementType *data; // 栈数据存储
    int memNum; // 栈元素个数
    int size; // 栈大小
};
```

例子3: 用类模板实现栈(2/6)

```
// tstack.h: 类模板的定义 (续)
template < typename ElementType >
Stack <ElementType>::Stack(int s)
{size = s > 0 ? s : 8;
 data = new ElementType[s];
 memNum = 0;
}
template < typename ElementType >
int Stack <ElementType>::pop(
                        ElementType &num)
{if (memNum==0)
 return 0;
 num = data[ -- memNum ];
 return 1;
}
```

例子3: 用类模板实现栈(3/6)

```
// tstack.h: 类模板的定义 (续)
template < typename ElementType >
int Stack <ElementType>::push(
                                ElementType mem)
{if (memNum == size)
    return 0;
    data[ memNum ++ ] = mem;
    return 1;
}
```

例子3: 用类模板实现栈(4/6)

```
//ex18_2.cpp:使用类模板
#include <iostream>
#include "tstack.h"
main()
{Stack <double>  doubleStack(6);
  // Stack <double> 得到模板类
  double f = 3.14;
  cout<<"Pushing elements into
          doubleStack:\n";
  while (doubleStack.push(f))
  {cout << f << ' ';
    f += f;
  }
  cout << "\nStack is full. Cannot push "
        << f << " onto the doubleStack.";
```

例子3: 用类模板实现栈(5/6)

```
cout<<"\nPopping elements
        from doubleStack:\n";
while (doubleStack.pop(f))
    cout << f << ' ';
cout<<"\nStack is empty.Cannot pop.\n";
Stack <int> intStack; //intStack是模板类对象
int i = 1;
cout<<"\nPushing elements
        into intStack:\n";
while (intStack.push(i))
{cout << i << ' ';
  ++i;
}
cout<<"\nStack is full.push "
    <<i<<"failed.";
```

例子3: 用类模板实现栈(6/6)

```
cout<<"\n\nPopping elements
      from intStack:\n";
while (intStack.pop(i))
    cout << i << ' ';
cout <<
    "\nStack is empty. Cannot pop.\n";

return 0;
}
```


程序执行结果

```
Pushing elements onto doubleStack:  
3.14 6.28 12.56 25.12 50.24 100.48  
Stack is full. push 200.96 failed.
```

```
Popping elements from doubleStack:  
100.48 50.24 25.12 12.56 6.28 3.14  
Stack is empty. Cannot pop.
```

```
Pushing elements onto intStack:  
1 2 3 4 5 6 7 8  
Stack is full. push 9 failed.
```

```
Popping elements from intStack:  
8 7 6 5 4 3 2 1  
Stack is empty. Cannot pop.
```

学习目的检测

- 了解类属程序设计的概念
- 理解模板和模板的实例化机制
- 掌握模板的定义和使用方法

Thank you !